

从 0 到 GitHub Pages： 一个网站项目的完整工程链路

从目标定义、本地开发、版本控制，到 GitHub Actions 自动部署与上线验证。本文以可复现、可审查、能交给团队执行为标准，梳理一条完整的网站上线链路。

Winston

V1.0 · 2026-05-05

技术长文 / GitHub Pages / Vite / GitHub Actions

| 目录

01	执行摘要	03
02	边界先于代码: GitHub Pages 适合承载什么	04
03	从空目录到可运行项目	06
04	写出能上线的网页	09
05	Git 与 GitHub: 把本地状态变成远端事实	12
06	用 GitHub Actions 部署到 Pages	15
07	上线验证、故障排查与维护	19
08	附录: 命令清单与来源	23

执行摘要

从 0 构建网站并部署到 GitHub，不是把几个命令串起来。一个稳定的网站项目至少要同时满足四个契约：**用户看到的页面契约**、本地可复现的构建契约、远端仓库的版本契约，以及上线后可验证的发布契约。

核心结论

- 如果网站没有构建步骤，GitHub Pages 可以直接发布静态文件；如果使用 Vite、React、Vue 或其他需要产物目录的工具，应使用 GitHub Actions 构建并上传产物。
- Vite 当前文档要求 Node.js 版本为 **20.19+** 或 **22.12+**；Vite 的默认生产产物目录是 `dist`，部署到项目页时必须配置正确的 `base` 路径。
- GitHub Pages 发布的是静态文件，不支持 PHP、Ruby、Python 等服务端语言；它适合文档、作品集、前端单页站、营销页和静态应用，不适合作为 SaaS 后端或敏感交易入口。
- 部署完成的标准包括本地预览、远端工作流、Pages URL、资源路径、可访问性与性能烟测全部通过。

本文采用的工程路径

本文把「从 0」定义为一个空目录开始：先建立一个可运行的 Vite 静态站点，再把它纳入 Git 管理，创建 GitHub 仓库，配置 GitHub Pages 的 Actions 部署，最后用可重复的检查清单确认上线质量。选择 Vite 是因为它的开发服务器、构建产物和部署文档覆盖了现代前端最常见的静态站点路径；如果你的站点只是一个手写 `index.html`，可以跳过 Node 与构建步骤，直接采用本文第 6 章的「无构建工作流」。

项目类型	推荐路径	关键判断
单页 HTML / CSS / JS	静态文件直接发布，或 Actions 上传根目录	没有依赖安装和构建产物，失败面最小。
Vite / React / Vue / Svelte	Actions 中 <code>npm ci</code> 、 <code>npm run build</code> 、上传 <code>dist</code>	构建产物才是可发布内容，源码目录不是最终站点。
Jekyll 文档站	GitHub Pages 内置 Jekyll 或 Actions	需要遵守 Jekyll 目录约定；自定义构建更适合团队。
需要后端 API、登录、数据库写入	Pages 只放前端；后端放到独立平台	Pages 是静态托管，不是应用服务器。

本文档回答的问题

空目录里先建什么文件? 为什么 GitHub Pages 有时显示空白页? `base` 应该写成什么? 什么时候用 `npm install` , 什么时候用 `npm ci` ? Actions 工作流里为什么要给 `pages: write` 和 `id-token: write` ? 上线后怎么判断这不是一次偶然成功?

边界先于代码：GitHub Pages 适合承载什么

部署方案先看运行时边界。GitHub Pages 的价值在于把静态文件托管、HTTPS、仓库版本和 Actions 自动化放在同一条链路上；它的边界也很清楚，不能把它当成一台可以执行任意服务端代码的服务器。

GitHub Pages 的基本事实

GitHub 官方文档明确说明，Pages 会发布仓库中的静态文件，可以配合静态站点生成器，也可以用自定义 GitHub Actions workflow 构建并发布站点。文档还说明，Pages 不支持 PHP、Ruby、Python 等服务端语言；这意味着你可以发布前端资源、预渲染页面和静态文档，但不能把登录态、数据库写入、文件上传处理或支付回调放在 Pages 本身执行。

事实	工程含义	来源口径
Pages 查找 index.html 、 index.md 或 README.md 作为入口	根目录或发布目录必须有明确入口文件；只有源码组件没有入口页面会导致发布后不可读。	GitHub Pages 创建站点文档
静态生成器可通过 Actions 构建并发布	Vite 这类需要构建的工具，应让 Actions 输出 dist 后再部署。	GitHub Pages / Vite 部署文档
Pages 不支持服务端语言	动态能力要交给外部 API、serverless 或独立应用平台；前端通过 HTTPS 调用。	GitHub Pages 创建站点文档
Pages 站点不应用于密码、信用卡等敏感交易	登录、支付、个人敏感信息采集需要更完整的安全边界和合规托管。	GitHub Pages HTTPS 与限制文档

项目页、用户页与路径问题

GitHub Pages 常见 URL 有两类。用户或组织站点通常是 `https://USERNAME.github.io/`，项目站点通常是 `https://USERNAME.github.io/REPO/`。差异会直接影响前端构建的资源路径。Vite 文档给出的规则很清楚：部署到用户站点或自定义域名时，`base` 可以是 `'/'`；部署到项目站点时，应设置为 `'/REPO/'`。如果忘记这一点，HTML 能打开，但 JS、CSS、图片会从错误路径加载，最终表现为白屏或样式丢失。

```
// vite.config.js
import { defineConfig } from 'vite'

export default defineConfig({
  // 项目页: https://USERNAME.github.io/hello-site/
  base: '/hello-site/',
})
```

发布源有两种，不要混用心智模型

GitHub Pages 可以从分支目录发布，也可以由 GitHub Actions 工作流上传构建产物。分支发布适合没有构建步骤的站点；Actions 发布适合需要安装依赖、生成产物、做额外校验的站点。团队项目优先采用 Actions，因为构建逻辑会被版本化，评审者能看到部署脚本，而不是依赖某个人在设置页里点过什么。

判断规则，如果仓库里的源码目录就是最终站点，分支发布足够；如果需要运行 `npm run build` 才能得到最终站点，就让 Actions 发布产物目录。不要把 `src`、`node_modules` 或未构建的框架源码当成 Pages 的发布内容。

容量、频率与商业边界

GitHub Pages 有明确的使用限制。官方限制文档列出：源仓库建议不超过 1 GB，发布站点不超过 1 GB，部署超时为 10 分钟，软带宽限制为每月 100 GB，分支构建软限制为每小时 10 次；自定义 Actions 工作流不受这个每小时 10 次构建限制。工程上可以把这些数字理解成边界条件：图片和视频不要直接塞进仓库，重资源走 CDN 或对象存储；高频商业流量不要把 Pages 当免费业务主站。

| 从空目录到可运行项目

本地项目的第一目标是让任何人能在一台干净机器上复现同一套开发、构建和预览流程。目录结构、Node 版本、包管理器、脚本名称和锁文件，比第一屏视觉更早决定项目是否可维护。

确认环境，不要猜

先检查工具版本。Vite 当前文档写明需要 Node.js **20.19+** 或 **22.12+**；如果项目模板提示需要更高版本，按模板提示升级。团队项目应把版本写进 README，也可以使用 `.node-version` 或包管理器字段减少口径漂移。

```
node --version
npm --version
git --version
gh --version
```

gh 是 GitHub CLI。它不是构建网站的必要条件，但能把创建仓库、添加远端、推送和查看 Actions 状态变成可脚本化流程。没有它也可以在网页端创建仓库，再用 `git remote add origin` 连接。

用 Vite 初始化项目

Vite 文档提供了交互式与非交互式两种初始化方式。教学场景建议先用交互式命令，理解模板选择；团队脚手架和文档示例建议给出非交互式命令，减少读者停在提示界面的概率。

```
# 交互式
npm create vite@latest

# 非交互式：创建 vanilla JavaScript 项目
npm create vite@latest hello-site -- --template vanilla

cd hello-site
npm install
npm run dev
```

如果你已经在目标目录里，可以使用 `.` 作为项目名。这里要先确认目录为空，避免脚手架覆盖已有文件。

```
mkdir hello-site
cd hello-site
npm create vite@latest . -- --template vanilla
```

理解每个脚本的职责

Vite 项目默认会在 `package.json` 中放置三个脚本：`dev` 启动开发服务器，`build` 生成生产产物，`preview` 在本地预览生产产物。Vite 文档特别说明，`vite preview` 用于本地检查构建结果，不应被当成生产服务器。

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "devDependencies": {
    "vite": "^7.0.0"
  }
}
```

命令	什么时候运行	判断结果
<code>npm run dev</code>	开发过程中	浏览器能访问本地开发服务器，改文件后页面能更新。
<code>npm run build</code>	提交前、CI 中、发布前	生成 <code>dist</code> ，没有构建错误、类型错误或资源缺失。
<code>npm run preview</code>	构建后	本地以静态方式打开 <code>dist</code> ，模拟远端静态托管。

目录结构应当早一点定下来

小项目不需要复杂分层，但需要让入口、样式、脚本、图片和公共静态资源有稳定位置。下面的结构足够覆盖大多数从 0 开始的静态站点。

```
hello-site/
  index.html
  package.json
  package-lock.json
  vite.config.js
  src/
    main.js
    styles.css
  public/
    favicon.svg
  .github/
    workflows/
      deploy.yml
  README.md
  .gitignore
```

`index.html` 在 Vite 中是项目入口，不是被藏在 `public` 里的模板。`public` 更适合放不经过打包处理、需要原样复制到产物目录的资源。业务脚本和样式放在 `src`，由入口模块导入。

锁文件是部署契约的一部分

本地安装依赖通常使用 `npm install`。CI 和部署环境应使用 `npm ci`，因为 `npm` 文档把它定义为面向自动化环境的干净安装：它要求存在 `package-lock.json`，当锁文件与 `package.json` 不一致时会失败，并且不会改写锁文件。这个行为很重要，失败比静默更新依赖更适合部署环境。

提交规则，提交 `package.json` 和 `package-lock.json`，不要提交 `node_modules`。部署环境根据锁文件重建依赖树，仓库不保存依赖目录本身。

写出能上线的网页

网站上线质量不只由构建工具决定。HTML 的语义、资源路径、键盘可用性、移动端布局 and 性能预算，会决定这个站点是一个能公开维护的工程项目，还是一个只在本机截图里成立的页面。

HTML 先表达结构，再交给 CSS 表达样式

MDN 对 HTML 的定义是网页内容的意义与结构。实践上，先写出正确的文档结构，再考虑视觉。页面通常至少需要语言、字符集、视口、标题、描述、主内容区域和合理的标题层级。下面这个骨架足够作为多数静态站点的起点。

```
<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>站点标题</title>
    <meta name="description" content="一句话说明这个页面提供什么。">
  </head>
  <body>
    <header>
      <nav aria-label="主导航">...</nav>
    </header>
    <main>
      <h1>页面主标题</h1>
      <section>...</section>
    </main>
    <footer>...</footer>
  </body>
</html>
```

可访问性不是后期插件。MDN 的可访问性指南强调，很多网页内容只要使用正确 HTML 元素就能获得浏览器内置的可访问性钩子。按钮用 `<button>`，导航用 `<nav>`，表单输入绑定 `<label>`，图片根据意义写 `alt`。这些都是结构，不是装饰。

场景	优先写法	避免
可点击命令	<code><button type="button"></code>	给 <code><div></code> 绑定点击事件却没有键盘语义。
站内跳转	<code></code>	用按钮模拟链接，导致复制链接与新标签打开失效。
表单输入	<code><label for="email"></code> + <code><input id="email"></code>	只用 <code>placeholder</code> 当标签。
图片	有意义图片写具体 <code>alt</code> ，装饰图可留空	把文件名、关键词堆砌或空泛描述写进 <code>alt</code> 。

CSS 从约束开始，而不是从视觉堆叠开始

可靠的 CSS 先约束布局边界：页面最大宽度、内容间距、颜色变量、文本行高、断点策略。不要一开始就用大量绝对定位拼画面，这会在移动端、长标题、不同系统字体下快速失效。一个稳健的起点如下。

```
:root {
  color-scheme: light;
  --bg: #f7f7f2;
  --text: #171717;
  --muted: #5f5f5a;
  --accent: #1b365d;
}

body {
  margin: 0;
  font-family: Charter, Georgia, "Source Han Serif SC", serif;
  color: var(--text);
  background: var(--bg);
  line-height: 1.55;
}

main {
  width: min(100% - 32px, 960px);
  margin: 0 auto;
}

img, svg, video {
  max-width: 100%;
  height: auto;
}
```

JavaScript 应当增强页面，而不是替代页面

从 0 建站时，最容易过度使用 JavaScript。可以先写出没有脚本也能阅读和跳转的 HTML，再把交互作为增强层加上去。这样做的好处是：构建失败面更小，首屏内容更稳定，搜索与辅助技术更容易理解页面，调试时也能快速判断问题发生在结构、样式还是脚本层。

性能指标要进入发布检查

Google 的 Web Vitals 文档把 Core Web Vitals 归纳为加载、交互和视觉稳定性三个方面，当前核心指标包括 LCP、INP 和 CLS，并建议以移动端和桌面端第 75 百分位为判断口径。对于一个刚上线的静态站，最低限度的工程动作是：减少首屏大图，给图片设置宽高或稳定容器，避免大量同步脚本阻塞主线程，发布后用 Chrome DevTools、PageSpeed Insights 或真实用户监测数据持续观察。

提交前页面检查

标题层级是否只有一个 h1？交互元素能否只用键盘操作？资源路径是否在 npm run preview 下也正确？移动端 375px 宽度是否没有横向滚动？图片是否有尺寸约束？这些问题比是否使用某个框架更接近上线质量。

| Git 与 GitHub: 把本地状态变成远端事实

Git 把项目状态切成可审查的版本。GitHub 在此之上提供远端仓库、权限、Actions、Pages、secret scanning 和发布记录。上线链路从第一次提交开始就应当可解释。

初始化仓库

先建立 `.gitignore`，再提交。这样不会把依赖目录、构建产物、环境变量和系统临时文件带进第一次提交。

```
cat > .gitignore <<'EOF'
node_modules/
dist/
.env
.env.*
!.env.example
.DS_Store
EOF
```

如果手动创建文件，不要用上面的重定向覆盖已有团队文件；它适合新项目。实际工程里可以直接编辑 `.gitignore`，保留项目已有规则。

```
git init
git add .
git commit -m "Initial website"
git branch -M main
```

创建 GitHub 仓库并推送

GitHub CLI 的 `gh repo create` 支持从当前目录创建远端仓库，并通过 `--source`、`--remote`、`--push` 把已有本地提交推送上去。非交互式命令要显式声明公开、私有或内部可见性。

```
gh auth status
gh repo create hello-site --public --source=. --remote=origin --push
```

如果你已经在网页端创建了仓库，使用 GitHub 文档中的远端流程即可：添加 `origin`，验证远端，再推送 `main`。

```
git remote add origin git@github.com:OWNER/hello-site.git
git remote -v
git push -u origin main
```

提交前做一次 secret 检查

GitHub push protection 会在推送时阻止已知类型的硬编码凭证进入仓库。官方文档说明，用户级 push protection 对 GitHub.com 公共仓库默认启用；仓库级保护可以由管理员在更高层级启用。不要把这个能力当成唯一防线。更稳妥的做法是从第一天开始使用 `.env` 保存本地密钥，用 `.env.example` 提供字段示例，把实际密钥放进部署平台的 Secrets。

文件	是否提交	说明
<code>.env</code>	否	本地真实密钥和环境配置。
<code>.env.local</code>	否	个人本地覆盖项。
<code>.env.example</code>	是	只写变量名和假值，帮助别人配置环境。
<code>package-lock.json</code>	是	部署复现依赖树需要它。
<code>dist/</code>	通常否	Actions 会从源码构建产物；除非项目明确采用提交产物的策略。

| 用 GitHub Actions 部署到 Pages

对于需要构建的网站，最稳的 Pages 路径是：推送源码到 `main`，Actions 在干净环境安装依赖并构建，上传静态产物，Pages 部署这个产物。这样本地机器不再是发布系统的一部分，发布逻辑被放进仓库接受评审。

先在 Pages 设置中选择 GitHub Actions

Vite 的静态部署文档要求在仓库 Settings → Pages 中，把 Build and deployment 的 Source 设为 GitHub Actions。GitHub Pages 自定义工作流文档也说明，使用自定义工作流前需要在当前仓库启用这种发布源。管理员可以通过网页设置完成，也可以使用 REST API 创建或更新 Pages 站点，把 `build_type` 设为 `workflow`。

```
# 创建 Pages 站点并使用 workflow 构建。OWNER 和 REPO 替换为真实值。
gh api \
  --method POST \
  repos/OWNER/REPO/pages \
  -f build_type=workflow
```

如果仓库已经启用 Pages，上面的请求可能返回冲突。此时使用更新接口。

```
gh api \
  --method PUT \
  repos/OWNER/REPO/pages \
  -f build_type=workflow
```

Vite 项目的部署工作流

下面的工作流采用 2026-05-05 核验到的官方 action 主版本：`actions/checkout@v6`、`actions/setup-node@v6`、`actions/configure-pages@v6`、`actions/upload-pages-artifact@v5`、`actions/deploy-pages@v5`。固定主版本比跟随 `main` 更稳；合规要求更高的环境可以进一步固定到完整版本号或提交 SHA。

```
name: Deploy static content to Pages

on:
  push:
    branches: ['main']
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: 'pages'
  cancel-in-progress: true

jobs:
  deploy:
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Set up Node
        uses: actions/setup-node@v6
        with:
          node-version: lts/*
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Build
        run: npm run build

      - name: Setup Pages
        uses: actions/configure-pages@v6

      - name: Upload artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: './dist'

      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v5
```

逐行理解工作流

片段	职责	常见错误
<code>on.push.branches</code>	只在默认分支推送时自动部署。	默认分支不是 <code>main</code> ，但工作流写死 <code>main</code> 。
<code>permissions</code>	给 <code>GITHUB_TOKEN</code> 最小部署权限。	缺少 <code>pages: write</code> 或 <code>id-token: write</code> ，部署步骤失败。
<code>npm ci</code>	按锁文件干净安装依赖。	没有提交 <code>package-lock.json</code> ，或锁文件与 <code>package.json</code> 不一致。
<code>npm run build</code>	生成生产静态文件。	本地只跑过 <code>dev</code> ，没有发现生产构建错误。
<code>upload-pages-artifact</code>	把 <code>dist</code> 打包成 Pages 可部署 artifact。	上传了仓库根目录或错误目录，导致页面入口缺失。
<code>deploy-pages</code>	把 artifact 部署到 <code>github-pages</code> 环境。	没有环境 URL 输出，或 Pages 发布源没有切到 Actions。

无构建静态站的工作流

如果项目只是手写静态文件，可以不用 Node。下面的工作流把根目录下需要发布的文件复制到 `_site`，再上传这个目录。复制步骤避免把仓库中的开发材料误发布出去。


```

name: Deploy document site to Pages

on:
  push:
    branches: ['main']
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: 'pages'
  cancel-in-progress: true

jobs:
  deploy:
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/configure-pages@v6
      - name: Prepare static site
        run: |
          mkdir -p _site
          cp index.html _site/
          cp -R assets _site/ 2>/dev/null || true
      - uses: actions/upload-pages-artifact@v5
        with:
          path: '_site'
      - id: deployment
        uses: actions/deploy-pages@v5

```

自定义域名放在部署之后

先让默认 `github.io` URL 稳定工作，再接入自定义域名。GitHub 文档建议先验证自定义域名，降低域名接管风险；HTTPS 文档说明，正确配置的 Pages 站点支持 HTTPS 与强制 HTTPS，但混合内容会导致资源加载和安全问题。工程顺序应是：默认域名通过 → 配置 DNS → 在 Pages 设置中加入域名 → 等待证书 → 开启 Enforce HTTPS → 全站搜索 `http://` 并改为 HTTPS。

上线验证、故障排查与维护

上线需要一组可重复的证据。每次发布都应能回答三个问题：构建产物是否正确，部署系统是否执行了预期工作流，用户实际访问的 URL 是否加载了预期资源。

本地验证

提交前先在本地跑生产构建和生产预览。开发服务器会掩盖一些路径和缓存问题，只有 `build` 后的预览更接近 Pages 环境。

```
npm run build
npm run preview
```

打开浏览器开发者工具，检查 Network 面板是否存在 404；检查 Console 是否有模块加载失败；在移动端宽度下确认没有横向滚动；刷新深层路由，如果是 SPA，需要确认路由策略与 Pages 的静态文件能力匹配。

远端验证

1. 打开 GitHub 仓库的 Actions 页，确认最新工作流运行成功。
2. 打开工作流详情，确认上传的目录是 `dist` 或你准备好的 `_site`，不是源码目录。
3. 打开 Settings → Pages，确认 Source 是 GitHub Actions，并记录 Pages 给出的 URL。
4. 访问 URL，强制刷新，检查 CSS、JS、图片路径是否全部为 200。
5. 修改一个可见文本，提交后再次观察 Actions 和线上页面，确认自动部署链路闭合。

常见故障表

现象	最可能原因	处理方式
页面打开但样式和脚本 404	Vite base 没有按项目页设置为 <code>/REPO/</code>	修改 <code>vite.config.js</code> ，重新构建部署。
Actions 中 <code>npm ci</code> 失败	缺少锁文件，或锁文件与依赖声明不一致	本地运行 <code>npm install</code> 更新锁文件并提交。
部署步骤权限失败	工作流缺少 Pages 或 OIDC 权限	在 workflow 顶层加入 <code>permissions: contents: read, pages: write, id-token: write</code> 。
Pages 仍显示旧内容	工作流未触发、浏览器缓存、访问了旧分支发布 URL	检查 Actions 最新 run、Pages 设置、浏览器强制刷新。
仓库根目录只有 README 被显示	发布源指向分支根目录而不是构建产物	切换到 GitHub Actions，上传 <code>dist</code> 。
自定义域名 HTTPS 未生效	DNS 尚未传播、证书还在签发、存在混合内容	等待 DNS，检查 Pages 证书状态，全站搜索 <code>http://</code> 。

发布后的维护节奏

稳定站点也需要维护。每月检查一次依赖更新和 action 主版本，每次修改后都跑本地构建；重要站点应启用分支保护，要求 PR 和 Actions 通过后再合并。性能上，用 PageSpeed Insights 或 Chrome DevTools 做实验室诊断，用真实用户数据观察线上体验。安全上，不在前端保存不可公开的密钥，不把 Pages 用作敏感交易入口，不把关闭的 Pages 站点继续绑定在 DNS 上。

完成定义

一个网站项目完成时，新人能按 README 在本地跑起来，CI 能从空环境构建，GitHub Pages 能由工作流发布，线上 URL 能被用户稳定访问，出了问题能通过日志和检查清单定位，而不依赖作者回忆当时点过哪些设置。

| 附录：命令清单与来源

A. 从 0 到上线的最短命令链

```
npm create vite@latest hello-site -- --template vanilla
cd hello-site
npm install
npm run dev

# 另开终端，确认生产构建
npm run build
npm run preview

git init
printf "node_modules/\ ndist/\ n.env\ n.env.*\ n!.env.example\ n.DS_Store\ n" > .gitignore
git add .
git commit -m "Initial website"
git branch -M main

gh auth status
gh repo create hello-site --public --source=. --remote=origin --push

# 仓库 Settings → Pages → Source → GitHub Actions
# 添加 .github/workflows/deploy.yml 后：
git add .github/workflows/deploy.yml
git commit -m "Deploy site with GitHub Pages"
git push
```

B. 发布检查清单

阶段	检查项
本地	npm run build 成功； npm run preview 下资源无 404；移动端无横向滚动。
版本	package-lock.json 已提交； node_modules 、 dist 、真实 .env 未提交。
远端	仓库有 origin ； main 已推送； Actions 工作流出现在仓库中。
Pages	发布源为 GitHub Actions； workflow 权限包含 contents: read 、 pages: write 、 id-token: write 。
上线	Pages URL 可访问； CSS/JS/图片为 200； HTTPS 生效；必要时自定义域名已验证。

C. 主要资料来源

- GitHub Docs: Creating a GitHub Pages site, 核验 Pages 入口、静态文件、服务端语言边界与发布时间说明。

- [GitHub Docs: Using custom workflows with GitHub Pages](#), 核验 `configure-pages`、`artifact` 上传与部署权限要求。
- [Vite Docs: Getting Started 与 Deploying a Static Site](#), 核验 Node 版本、脚手架命令、默认脚本、`dist` 产物与 [GitHub Pages base](#) 配置。
- [npm Docs: npm ci 与 package.json](#), 核验锁文件和脚本约定。
- [MDN: HTML 与 HTML: A good basis for accessibility](#), 核验语义 HTML 与可访问性基础。
- [web.dev: Web Vitals](#), 核验 Core Web Vitals 指标口径。
- [GitHub CLI Manual: gh repo create](#), 核验从本地目录创建远端仓库并推送的参数。

D. 资料口径

本文资料核验日期为 **2026-05-05**。GitHub Actions 官方 action 主版本、Vite Node 版本要求、GitHub Pages REST API 字段和 Pages 限制属于会变化的事实；实际用于生产前，应重新打开官方文档核对。本文不代表 GitHub、Vite、npm、MDN 或 Google 官方立场。